LAPORAN PRAKTIKUM

PEMROGRAMAN BERORIENTASI OBJEK

MODUL 07 - Abstract dan Interface



CUT GEUBRINA DWI MAHARANI

103132400034

S1SD-05-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Dasar teori utama yang mendasari program Sistem Kasir Terpadu Bioskop ini meliputi Abstraksi untuk menyembunyikan detail kompleks melalui abstract class yang tidak dapat diinstansiasi secara langsung, Kontrak Antarmuka (Interface) sebagai standar operasional lintas entitas yang menyatukan silsilah kelas berbeda, serta pemanfaatan fitur modern Java 8+ seperti default method untuk pewarisan logika siap pakai dan static method untuk fungsi utilitas kasir. Seluruh konsep tersebut bermuara pada penerapan Polimorfisme melalui array heterogen bertipe interface, yang memungkinkan sistem memproses berbagai objek produk berbeda secara dinamis dalam satu keranjang belanja menggunakan pemanggilan metode yang seragam.

B. Guided

1. [Guided 1]

```
package guided.gd1;


// Fondasi umum: belum punya peran spesifik, jadi tidak boleh diinstansiasi langsung
public abstract class Employee {
    private String name;
    public Employee(String name){
        this.name = name;
    }
    public String getName(){
        return name;
    }
    // setiap pegawai pasti punya gaji tapi rumusnya berbeda
    public abstract double earnings();
}
```

```
package guided.gd1;


// bentuk konkret
```

```java
public class SalariedEmployee extends Employee {

    private double weeklySalary;


    public SalariedEmployee(String name, double weeklySalary){

        super(name);

        this.weeklySalary = weeklySalary;

    }


    @Override

    public double earnings(){

        return weeklySalary;

    }

}
```

```java
package guided.gd1;


public class Main {

    public static void main(String[] args) {

        SalariedEmployee salaried = new
SalariedEmployee("budi", 5000000);

        System.out.println("Nama : "+ salaried.getName());

        System.out.println("gaji : Rp" + salaried.earnings());

    }

}
```

Output :

```
Nama : budi
gaji : Rp5000000.0
```

2. [Guided 2]

```java
package guided.gd2;

// Fondasi umum: belum punya peran spesifik, jadi tidak boleh diinstansiasi langsung
public abstract class Employee implements Payable {
   private String name;

    public Employee(String name){
        this.name = name;
    }

    public String getName(){
        return name;
    }

    // setiap pegawai pasti punya gaji, tetapi dengan rumus yang berbeda-beda.
    public abstract double earnings();

    // saat ada penagihan, alihkan langsung ke metode kalkulasi gaji (earning)
    @Override
    public double getPaymentAcount(){
        return earnings();
    }
}
```

```java
package guided.gd2;

public class Invoice implements Payable {
    private String partName;
    private int quantity;
    private double pricePerItem;

    public Invoice(String partName, int quantity, double pricePerItem){
        this.partName = partName;
        this.quantity = quantity;
        this.pricePerItem = pricePerItem;
    }
```

```
    public String getpartName(){
        return partName;
    }

    @Override
    public double getPaymentAcount(){
        return quantity * pricePerItem;
    }

}
```

```
package guided.gd2;

public interface Payable {
    double getPaymentAcount();
}
```

```
package guided.gd2;

// bentuk konkret
public class SalariedEmployee extends Employee {
    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary){
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings(){
        return weeklySalary;
    }
}
```

```
package guided.gd2;

public class Main {
    public static void main(String[] args) {
        Payable[] items = new Payable[2];
        items[0] = new Invoice("Monitor LED", 2, 1500000);
```

```
        items[1] = new SalariedEmployee("Budi", 5000000);

        // Dynamic dispatch : java otomatis mengeksekusi rumus
yang tepat sesuai objek

        for (Payable item: items){
            System.out.println("jumlah yang harus dibayar: Rp"
+ item.getPaymentAcount());
        }
    }
}
```

Output:

```
jumlah yang harus dibayar: Rp3000000.0
jumlah yang harus dibayar: Rp5000000.0
```

3. [Guided 3]

```
package guided.gd3;

public abstract class Employee implements Payable {
      private String name;

    public Employee(String name){
        this.name = name;
    }

    public String getName(){
        return name;
    }

    // setiap pegawai pasti punya gaji, tetapi dengan rumus
yang berbeda-beda.
    public abstract double earnings();

    // saat ada penagihan, alihkan langsung ke metode kalkulasi
gaji (earning)
    @Override
     public double getPaymentAcount(){
```

```
        return earnings();
    }
}
```

```
package guided.gd3;

public class Invoice implements Payable {
     private String partName;
    private int quantity;
    private double pricePerItem;

    public Invoice(String partName, int quantity, double
pricePerItem){
        this.partName = partName;
        this.quantity = quantity;
        this.pricePerItem = pricePerItem;
    }

    public String getPartName(){
        return partName;
    }

    @Override
    public double getPaymentAcount(){
        return quantity * pricePerItem;
    }

}
```

```
package guided.gd3;

public interface Payable {
    double getPaymentAcount();

    default void printReceipt(){
        System.out.println("Struk Resmi - Total: Rp " +
getPaymentAcount());
    }

    // fungsi utilitas
    static void logTransaction(String message){
```

```
        System.out.println("[AUDIT LOG]" + message);
    }
}
```

```
package guided.gd3;

public class SalariedEmployee extends  Employee {
     private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary){
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings(){
        return weeklySalary;
    }
}
```

```
package guided.gd3;

public class Main {
     public static void main(String[] args) {
        Payable.logTransaction(" Sistem pembayaran
diinisialisasi.");

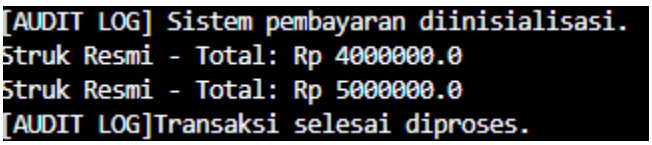
        Payable[] items = new Payable[2];
        items[0] = new Invoice("Ram DDR 5 16GB", 2, 2000000);
        items[1] = new SalariedEmployee("Budi", 5000000);

        for (Payable item : items){
            item.printReceipt();
        }

        Payable.logTransaction("Transaksi selesai diproses.");

    }
}
```

Output:



C. Unguided

Soal Unguided — Modul 7: Pemrograman Berorientasi Objek

Abstraksi, Kontrak Interface, dan Polimorfisme Heterogen

Petunjuk Umum

Buatlah program Java berdasarkan skenario sistem kasir bioskop berikut. Rancang kelas dan antarmuka dengan mematuhi prinsip OOP yang telah dipelajari.

- **Struktur Berkas Pengerjaan (`unguided/`)** Seluruh kelas dan antarmuka wajib berada di dalam direktori `unguided/` dengan rincian berkas berikut (satu file `.java` per kelas/interface):

```
None
unguided/
├── ItemTransaksi.java  (Interface)
├── TiketBioskop.java   (Abstract Class)
├── TiketReguler.java   (Concrete Subclass)
├── TiketVIP.java       (Concrete Subclass)
├── PaketSnack.java     (Concrete Class)
└── Main.java           (Kelas Eksekusi Utama)
```

  Pastikan baris paling atas di setiap berkas mencantumkan package yang sesuai (contoh: `package com.[nama_panggilan].unguided;`).

- **Enkapsulasi:** Seluruh *field* (atribut) wajib berstatus `private` dan hanya boleh diakses melalui *getter* oleh kelas di luar dirinya.
- **Keyword this:** Gunakan parameter dengan nama yang sama dengan nama *field* pada *constructor*, sehingga wajib menggunakan kata kunci `this` secara eksplisit.

- **Koleksi Statis:** Dilarang menggunakan tipe data koleksi dinamis seperti `ArrayList`. Gunakan array statis standar.

Soal — Sistem Kasir Terpadu Bioskop (*Cinema POS*)

Sebuah bioskop memerlukan sistem kasir yang mampu memproses pembayaran dalam satu keranjang belanja. Di kasir, penonton tidak hanya membeli **Tiket Nonton**, tetapi juga bisa membeli makanan/minuman (**Paket Snack F&B**).

Secara konsep sistem, **Tiket Bioskop** dan **Paket Snack** berasal dari silsilah keluarga (hierarki) yang sepenuhnya berbeda. Tiket film murni pun bersifat abstrak (tidak bisa dibeli tanpa jenis studio). Namun, di mata program kasir, tiket dan makanan harus bisa disatukan, dihitung total tagihannya, dan dicetak di dalam satu struk secara otomatis.

Pengerjaan soal ini dibagi menjadi 4 tugas utama untuk melatih abstraksi kelas, antarmuka (*interface*), dan fitur modern Java 8+.

Tugas 1: Fondasi Kelas Abstrak (*Abstract Class*)

Buatlah hierarki kelas tiket bioskop dengan spesifikasi berikut:

1. Kelas Induk `TiketBioskop` (Wajib `abstract`)

- **Atribut (Private):** `judulFilm` (`String`), `hargaDasar` (`double`), `jumlahKursi` (`int`).
- **Konstruktor:** Menerima dan menginisialisasi ketiga atribut tersebut.
- **Method:** Buat *getter* untuk ketiga atribut.
- **Abstract Method:**
  - `public abstract double hitungTotalBayar();`
  - `public abstract String getNamaTransaksi();` *(Beban perumusan kedua metode ini diserahkan kepada kelas turunannya agar output di struk nanti spesifik untuk tiap jenis tiket).*

2. Kelas Turunan `TiketReguler`

- **Atribut Tambahan (Private):** `nomorStudio` (`int`).
- **Konstruktor:** Menginisialisasi atribut induk via `super(judulFilm, hargaDasar, jumlahKursi)` dan atribut lokal `nomorStudio`.
- **Override Method:**

    - `hitungTotalBayar()`: Mengembalikan hasil `hargaDasar * jumlahKursi`.
    - `getNamaTransaksi()`: Mengembalikan string `"Tiket Reguler (Studio " + nomorStudio + ") - " + getJudulFilm() + " (" + getJumlahKursi() + " x Rp" + getHargaDasar() + ")"`.

3. Kelas Turunan `TiketVIP`

- **Atribut Tambahan (Private):** `biayaFasilitasVIP` (`double`). *(Catatan: Biaya fasilitas seperti sofa recliner ini dikenakan untuk **setiap kursi**).*
- **Konstruktor:** Menginisialisasi atribut induk via `super(...)` dan atribut lokal `biayaFasilitasVIP`.
- **Override Method:**
    - `hitungTotalBayar()`: Mengembalikan hasil `(hargaDasar + biayaFasilitasVIP) * jumlahKursi`.
    - `getNamaTransaksi()`: Mengembalikan string `"Tiket VIP Premiere - " + getJudulFilm() + " (" + getJumlahKursi() + " x Rp" + (getHargaDasar() + biayaFasilitasVIP) + ")"`.

Tugas 2: Kontrak Antarmuka Lintas Entitas (*Interface*)

Sistem kasir membutuhkan standar publik agar setiap barang bisa diproses.

1. Antarmuka `ItemTransaksi` Buatlah *interface* bernama `ItemTransaksi` dengan dua deklarasi kontrak (metode abstrak):

- `double getNominalBayar();`
- `String getNamaTransaksi();`

2. Integrasi Kelas `TiketBioskop` ke Antarmuka

- Tambahkan `implements ItemTransaksi` pada deklarasi kelas `TiketBioskop`.
- **Penting (Alasan Arsitektural):** Metode `hitungTotalBayar()` adalah logika internal spesifik tiket, sedangkan `getNominalBayar()` adalah kontrak publik kasir. Wujudkan metode `getNominalBayar()` di dalam `TiketBioskop` yang isinya sekadar me-*return* nilai dari `hitungTotalBayar()`.
- *(Catatan: Anda tidak perlu menata ulang `getNamaTransaksi()` di sini karena ia sudah dideklarasikan sebagai metode abstrak pada Tugas 1).*

3. Kelas Baru: `PaketSnack` (Benda Non-Tiket) Buat kelas independen bernama `PaketSnack` yang **bukan** turunan `TiketBioskop`, tetapi mengimplementasikan `ItemTransaksi`.

- **Atribut (Private):** `namaPaket` (`String`), `jumlahPorsi` (`int`), `hargaPerPorsi` (`double`).
- **Konstruktor:** Menginisialisasi ketiga atribut.
- **Override Method:**
  - `getNominalBayar()`: Mengembalikan hasil `jumlahPorsi * hargaPerPorsi`.
  - `getNamaTransaksi()`: Mengembalikan string `"F&B Snack: " + namaPaket + " (" + jumlahPorsi + " x Rp" + hargaPerPorsi + ")"`.

Tugas 3: Modernisasi Antarmuka Java 8+

Suntikkan fitur tambahan ke antarmuka `ItemTransaksi` tanpa mengharuskan kelas-kelas di atas dimodifikasi ulang:

1. Default Method Tambahkan metode `default void cetakRincianItem()` yang bertugas mencetak format baris struk ke layar. Panggil metode milik antarmuka (`getNamaTransaksi` dan `getNominalBayar`) untuk mengambil data secara dinamis. Format wajib: `- [Nama Transaksi (Kuantitas x Harga Satuan)] | Total: Rp[Nominal Bayar]`

2. Static Method Tambahkan metode `static void catatLogKasir(String pesan)` yang mencetak teks: `[LOG SISTEM KASIR] [pesan]`

Tugas 4: Pengujian Polimorfisme di `Main`

Tuliskan alur berikut di dalam *method* `main`:

1. **Uji Proteksi Abstraksi:** Tuliskan baris kode instansiasi `TiketBioskop` (misal: `new TiketBioskop(...)`), lalu jadikan baris tersebut sebagai **komentar** (`//`).
2. **Buka Kasir:** Panggil metode statis `catatLogKasir("Membuka transaksi baru untuk Pelanggan #042.");` langsung melalui nama antarmukanya.
3. **Buat Objek Transaksi:**

- 1 objek `TiketReguler` (Film `"Interstellar"`, harga dasar 40000, 2 kursi, Studio 3).
- 1 objek `TiketVIP` (Film `"Dune: Part Two"`, harga dasar 60000, 2 kursi, biaya VIP 25000).
- 1 objek `PaketSnack` (`"Combo Popcorn Caramel + 2 Soda"`, 1 porsi, harga 55000).

4. **Array Polimorfik:** Simpan ketiga objek tersebut ke dalam satu array heterogen bertipe `ItemTransaksi[]`.
5. **Pemrosesan Dinamis (Loop):**
   - Gunakan perulangan (*for* atau *for-each*) terhadap array tersebut.
   - Di dalam perulangan, panggil *default method* `cetakRincianItem()` **lewat masing-masing objek/elemen array**.
   - Hitung dan jumlahkan nominal bayar keseluruhan belanjaan secara akumulatif.
6. **Tutup Kasir:** Tampilkan grand total seluruh belanjaan, lalu panggil kembali metode statis `catatLogKasir("Transaksi #042 selesai diproses.");`.

Contoh Output yang Diharapkan

```
None
[LOG SISTEM KASIR] Membuka transaksi baru untuk Pelanggan #042.
=== DAFTAR TRANSAKSI BIOSKOP ===
- Tiket Reguler (Studio 3) - Interstellar (2 x Rp40000.0) | Total: Rp80000.0
- Tiket VIP Premiere - Dune: Part Two (2 x Rp85000.0) | Total: Rp170000.0
- F&B Snack: Combo Popcorn Caramel + 2 Soda (1 x Rp55000.0) | Total: Rp55000.0
------------------------------------------------------------
TOTAL TRANSAKSI : Rp305000.0
============================================================
[LOG SISTEM KASIR] Transaksi #042 selesai diproses.
```

Jawaban :

1. ItemTransaksi.java

```java
package unguided;

public interface ItemTransaksi {
    double getNominalBayar();
    String getNamaTransaksi();
```

```
    // Tugas 3: Default Method
    default void cetakRincianItem() {
        System.out.println("- " + getNamaTransaksi() + " |
Total: Rp" + getNominalBayar());
    }

    // Tugas 3: Static Method
    static void catatLogKasir(String pesan) {
        System.out.println("[LOG SISTEM KASIR] " + pesan);
    }
}
```

2. PaketSnack.java

```
package unguided;

public class PaketSnack implements ItemTransaksi {
    private String namaPaket;
    private int jumlahPorsi;
    private double hargaPerPorsi;

    public PaketSnack(String namaPaket, int jumlahPorsi, double
hargaPerPorsi) {
        this.namaPaket = namaPaket;
        this.jumlahPorsi = jumlahPorsi;
        this.hargaPerPorsi = hargaPerPorsi;
    }

    public String getNamaPaket() {
        return namaPaket;
    }

    public int getJumlahPorsi() {
        return jumlahPorsi;
    }

    public double getHargaPerPorsi() {
        return hargaPerPorsi;
    }
```

```
    @Override
    public double getNominalBayar() {
        return jumlahPorsi * hargaPerPorsi;
    }

    @Override
    public String getNamaTransaksi() {
        return "F&B Snack: " + namaPaket + " (" + jumlahPorsi +
" x Rp" + hargaPerPorsi + ")";
    }
}
```

3. TiketBioskop.Java

```
package unguided;

public abstract class TiketBioskop implements ItemTransaksi {
    private String judulFilm;
    private double hargaDasar;
    private int jumlahKursi;

    public TiketBioskop(String judulFilm, double hargaDasar,
int jumlahKursi) {
        this.judulFilm = judulFilm;
        this.hargaDasar = hargaDasar;
        this.jumlahKursi = jumlahKursi;
    }

    public String getJudulFilm() {
        return judulFilm;
    }

    public double getHargaDasar() {
        return hargaDasar;
    }

    public int getJumlahKursi() {
        return jumlahKursi;
    }
```

```
    public abstract double hitungTotalBayar();

    @Override
    public double getNominalBayar() {
        return hitungTotalBayar();
    }

    @Override
    public abstract String getNamaTransaksi();
}
```

4. TiketReguler.java

```
package unguided;

public class TiketReguler extends TiketBioskop {
    private int nomorStudio;

    public TiketReguler(String judulFilm, double hargaDasar,
int jumlahKursi, int nomorStudio) {
        super(judulFilm, hargaDasar, jumlahKursi);
        this.nomorStudio = nomorStudio;
    }

    public int getNomorStudio() {
        return nomorStudio;
    }

    @Override
    public double hitungTotalBayar() {
        return getHargaDasar() * getJumlahKursi();
    }

    @Override
    public String getNamaTransaksi() {
        return "Tiket Reguler (Studio " + nomorStudio + ") - "
+ getJudulFilm() + " (" + getJumlahKursi() + " x Rp" +
getHargaDasar() + ")";
    }
}
```

5. TiketVIP.java

```
package unguided;

public class TiketVIP extends TiketBioskop {
    private double biayaFasilitasVIP;

    public TiketVIP(String judulFilm, double hargaDasar, int
jumlahKursi, double biayaFasilitasVIP) {
        super(judulFilm, hargaDasar, jumlahKursi);
        this.biayaFasilitasVIP = biayaFasilitasVIP;
    }

    public double getBiayaFasilitasVIP() {
        return biayaFasilitasVIP;
    }

    @Override
    public double hitungTotalBayar() {
        return (getHargaDasar() + biayaFasilitasVIP) *
getJumlahKursi();
    }

    @Override
    public String getNamaTransaksi() {
        double totalHargaSatuan = getHargaDasar() +
biayaFasilitasVIP;
        return "Tiket VIP Premiere - " + getJudulFilm() + " ("
+ getJumlahKursi() + " x Rp" + totalHargaSatuan + ")";
    }
}
```

6. Main.java

```
package unguided;

public class Main {
    public static void main(String[] args) {
        // 1. Uji Proteksi Abstraksi (Tidak bisa diinstansiasi
langsung, harus dikomentari)
```

```
        // TiketBioskop errorTest = new TiketBioskop("Test",
50000, 1);

        // 2. Buka Kasir
        ItemTransaksi.catatLogKasir("Membuka transaksi baru
untuk Pelanggan #042.");

        // 3. Buat Objek Transaksi
        TiketReguler tiketReg = new
TiketReguler("Interstellar", 40000, 2, 3);
        TiketVIP tiketVip = new TiketVIP("Dune: Part Two",
60000, 2, 25000);
        PaketSnack snack = new PaketSnack("Combo Popcorn
Caramel + 2 Soda", 1, 55000);

        // 4. Array Polimorfik (Menggunakan array statis, bukan
ArrayList)
        ItemTransaksi[] keranjangBelanja = { tiketReg,
tiketVip, snack };

        // 5. Pemrosesan Dinamis
        System.out.println("=== DAFTAR TRANSAKSI BIOSKOP ===");
        double grandTotal = 0;

        for (int i = 0; i < keranjangBelanja.length; i++) {
            keranjangBelanja[i].cetakRincianItem();
            grandTotal +=
keranjangBelanja[i].getNominalBayar();
        }

        // 6. Tutup Kasir & Cetak Grand Total

System.out.println("----------------------------------------
------------------");
        System.out.println("TOTAL TRANSAKSI : Rp" +
grandTotal);

System.out.println("========================================
==================");
```

```
        ItemTransaksi.catatLogKasir("Transaksi #042 selesai
diproses.");
    }
}
```

Output :

```
[LOG SISTEM KASIR] Membuka transaksi baru untuk Pelanggan #042.
=== DAFTAR TRANSAKSI BIOSKOP ===
- Tiket Reguler (Studio 3) - Interstellar (2 x Rp40000.0) | Total: Rp80000.0
- Tiket VIP Premiere - Dune: Part Two (2 x Rp85000.0) | Total: Rp170000.0
- F&B Snack: Combo Popcorn Caramel + 2 Soda (1 x Rp55000.0) | Total: Rp55000.0
------------------------------------------------------------
TOTAL TRANSAKSI : Rp305000.0
============================================================
[LOG SISTEM KASIR] Transaksi #042 selesai diproses.
```

Penjelasan :

Program ini menerapkan tiga pilar utama OOP: Abstraksi, interface, dan Polimorfisme

1. ItemTransaksi.Java

   Interface ini berfungsi sebagai standar operasional bagi semua produk yang bisa dibeli di kasir. Kelas apa pun yang menggunakan interface ini wajib menyiapkan metode yang tertulis di dalamnya.

   - getNominalBayar() & getNamaTransaksi(): Memaksa kelas lain untuk mengisi rumus harga dan nama produknya sendiri
   - Default Method (cetakRincianItem()): Fitur Java 8 di mana interface bisa langsung punya kode isi. Fungsinya mencetak format struk standar secara otomatis
   - Static Method (catatLogKasir()): Fitur helper yang bisa dipanggil langsung dari nama interface-nya tanpa perlu membuat objek produk terlebih dahulu, digunakan untuk mencatat riwayat log sistem kasir

2. TiketBioskop.Java

   Kelas ini bertindak sebagai kerangka dasar untuk semua jenis tiket bioskop. Sesuai sifat abstract, kelas ini tidak bisa dibuat objeknya secara langsung (new TiketBioskop dilarang) karena masih bersifat umum.

   - Enkapsulasi (private & getter): Atribut dikunci dengan hak akses private agar aman dan hanya bisa dibaca melalui mode getter
   - Penghubung: Kelas ini mengimplementasikan ItemTransaksi. Ia mewariskan metode getNominalBayar() dengan cara memanggil hitungTotalBayar() rumus hitungnya sengaja dibiarkan abstrak agar nanti diselesaikan oleh kelas anak

3. TiketReguler.Java & TiketVIP.Java
   Kedua kelas ini adalah wujud nyata (konkret) dari turunan TiketBioskop. Di sinilah logika perhitungan harga spesifik ditulis melalui proses @Override:
   - TiketReguler: Menghitung total harga murni berdasarkan perkalian standar (hargaDasar * jumlahKursi) serta mencetak keterangan nomor studio reguler
   - TiketVIP: Memiliki tambahan atribut biayaFasilitasVIP. Rumus totalnya menjumlahkan harga dasar dengan biaya VIP terlebih dahulu sebelum dikalikan jumlah kursi

4. PaketSnack.Java
   Kelas ini dibuat untuk membuktikan fleksibilitas dari sebuah interface
   - Walaupun PaketSnack bukan anak dari TiketBioskop (silsilah keluarganya berbeda), ia tetap bisa masuk ke keranjang kasir yang sama karena sama-sama menandatangani kontrak implements ItemTransaksi.
   - Menghitung total harga makanan/minuman dari jumlahPorsi * hargaPerPorsi.

5. Main.Java
   Kelas utama tempat program dijalankan untuk mensimulasikan transaksi kasir:
   - Uji Proteksi: Membuktikan bahwa baris kode new TiketBioskop(...) akan memicu error jika dibuka kuncinya, karena kelas abstrak tidak boleh di-*instansiasi* langsung.
   - Log Awal: Memanggil fungsi log statis untuk mencatat status awal kasir bioskop.
   - Array Polimorfik (ItemTransaksi[]): Inilah inti dari konsep Polimorfisme. Kita cukup membuat satu buah keranjang belanja bertipe ItemTransaksi[] yang di dalamnya bisa menampung berbagai macam objek berbeda (Tiket Reguler, Tiket VIP, dan Paket Snack) secara berdampingan.
   - Perulangan Struk (Looping): Program melompat ke setiap item di dalam array secara otomatis, memanggil fungsi cetakRincianItem(), lalu menjumlahkan seluruh biayanya ke dalam variabel penampung grandTotal.
   - Log Akhir: Mencetak total keseluruhan uang yang harus dibayar oleh pelanggan lalu menutup sesi transaksi kasir.

D. Kesimpulan
   Kesimpulan dari praktikum ini adalah mahasiswa berhasil mengimplementasikan konsep Abstraksi menggunakan abstract class sebagai kerangka dasar yang aman dan wajib diturunkan, serta Kontrak Interface yang mampu menyatukan entitas dari silsilah keluarga berbeda ke dalam satu standar kasir yang sama. Selain itu, praktikum ini melatih pemanfaatan fitur modern Java 8+ seperti default method dan static method untuk mengoptimalkan struktur kode tanpa merusak kelas yang ada. Seluruh konsep tersebut

terangkum sempurna melalui penerapan Polimorfisme menggunakan array heterogen bertipe interface, yang memungkinkan sistem memproses berbagai objek produk berbeda secara dinamis, melakukan perulangan struk, serta menghitung total transaksi secara efisien dalam satu keranjang belanja.